

Design Principles of Programming Languages

Slot-H

Topics to be covered

- Evolution of Programming Languages
- Compilation and Interpretation
- Bootstrapping
- Data types and their implementation – Primitive and Composite
- Heap storage and heap storage management
- Garbage collection techniques
- Names - Binding, Lifetime, Scope
- Control Flow - Expression Evaluation, Assignment, Conditional, Loop statements
- Function calls and their implementation
- Non-local variable access
- Exceptions and Exception Handling
- Modular Programming
- Object Oriented Programming and its implementation details

Books

- Programming Language Pragmatics - By Michael Scott, Elsevier -Third Edition
- Programming Languages Design and Implementation by T. W. Pratt, Pearson- Fourth Edition

Introduction

□ Why are there so many programming languages?

□ Evolution

□ We've learned better ways of doing things over time

□ Special purposes

– Languages were designed for a specific problem domain

- Icon and Awk are good for manipulating character strings

- C is good for low-level systems programming.

- Prolog is good for reasoning about logical relationships among data.

Personal Preference

- Different people like different things.
- C -some people love the briefness of C; some do not like it.
- Some people think recursively; others prefer iteration.
- Some people like to work with pointers; others do not like java.

No one is standard and universally acceptable programming language.

What makes a language successful?

- *Expressive Power*

- All languages are equally powerful in technical sense (i.e. Turing complete)
- Language features have a huge impact on the programmer's ability to read, write, maintain, and analyze programs
- Abstraction facilities enhance expressive power

- *Ease of Use for Novice*

- Low learning curve and often interpreted, eg. C, Java and python

- *Ease of Implementation*

- Runs on virtually everything, e.g. Basic_, Pascal_ and Java

- *Excellent Compilers*

- Fortran has extremely good compilers (because it lacks recursion and pointers) and is therefore popular for numerical applications

- *Economics*

- Powerful sponsor: Cobol, Ada : U.S Department of defence.
- Some languages remain widely used long after "better" alternatives because of a huge base of installed software and programmer experience.

Why study programming languages?

- Improve your ability to develop effective algorithm.
 - Improving ease of existing programming language.
 - Increase vocabulary of useful programming construct.
 - Better choice of Programming language.
 - Easier to learn new language.
 - Easier to design new language.
- 
- A decorative graphic on the right side of the slide, consisting of several overlapping curved shapes in orange, light blue, and dark blue, creating a modern, abstract design.

Language types/Programming Paradigms

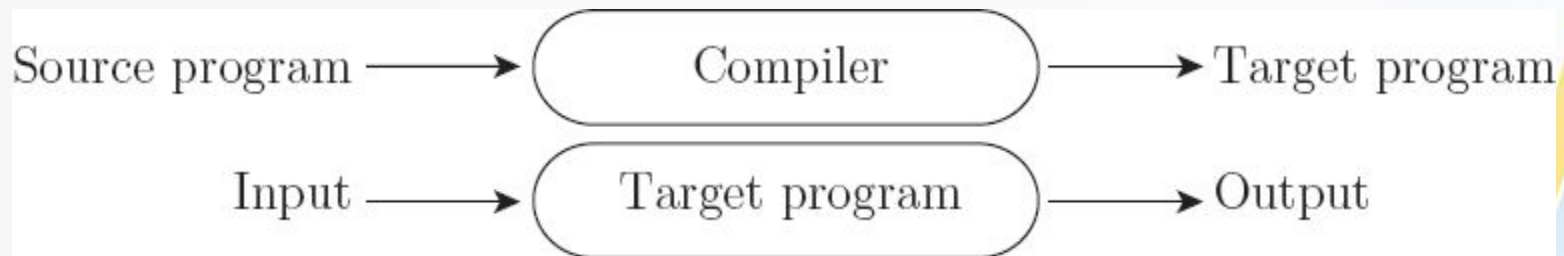
- Imperative languages, particularly the von Neumann languages, predominate
- The focus is on how the computer should do the things - more detailed
- In declarative languages, the focus is on what the computer is to do -abstracted.

Language Types

- Group languages as
 - imperative
 - object-oriented (Smalltalk, Eiffel, C++)
 - scripting languages (Perl, JavaScript, PHP)
 - declarative
 - functional (Scheme, ML, pure Lisp, FP)
 - logic, constraint-based (Prolog, VisiCalc, RPG)

Compilation vs. Interpretation

- Compilation vs. interpretation
- Pure Compilation
 - The compiler translates the high-level source program into an equivalent target program (typically in machine language), and then goes away: Ex. gcc/g++



Compilation vs. Interpretation

- Pure Interpretation
 - Interpreter stays around for the execution of the program
 - Interpreter is the locus of control during execution
 - Ex. Bourne Again SHell (BASH), python, perl



Compilation vs. Interpretation

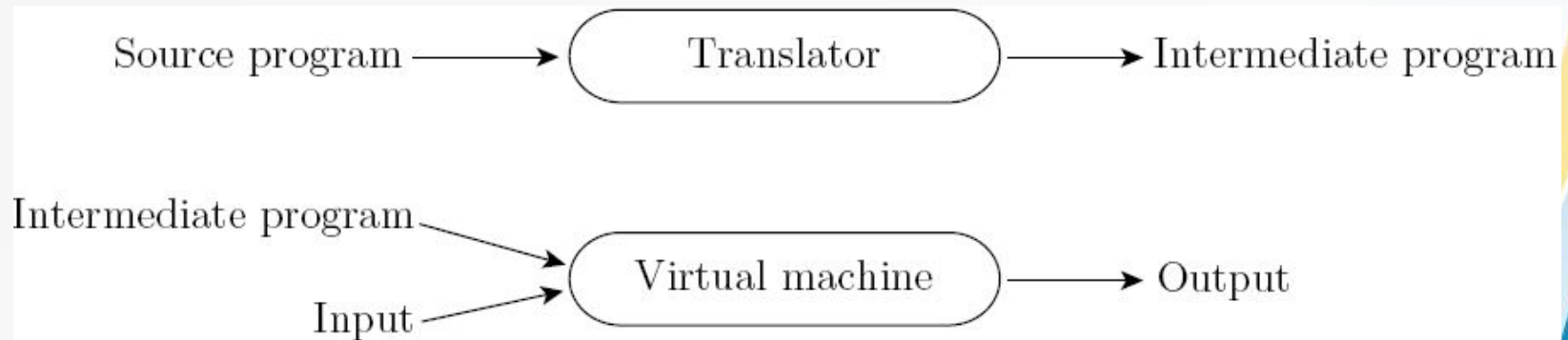
- Interpretation:
 - Greater flexibility
 - Better diagnostics (error messages)
- Compilation
 - Better performance
 - Everything done before run time

Compilation vs. Interpretation

- Note that compilation does NOT have to produce machine language for some sort of hardware.
- Compilation is *translation* from one language into another, with full analysis of the meaning of the input.

Compilation vs. Interpretation

- Common case is compilation or simple pre-processing, followed by interpretation
- Most language implementations include a mixture of both compilation and interpretation



Compilation vs. Interpretation

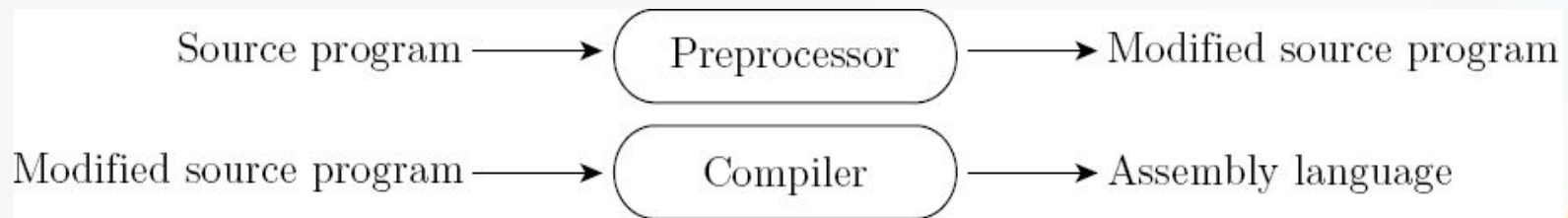
- Implementation strategies:

1. Preprocessor

- Removes comments and white space
- Groups characters into *tokens* (keywords, identifiers, numbers, symbols)
- Expands abbreviations in the style of a macro assembler

Compilation vs. Interpretation

- Implementation strategies:
 - The C Preprocessor (conditional compilation)
 - Preprocessor deletes portions of code, which allows several versions of a program to be built from the same source



Compilation vs. Interpretation

- The source code is read from the file and given to the preprocessor where it is translated into a modified source code file which is then given to the compiler for translation to machine language.
- The transformations performed by the preprocessor are directed by lines in the original source file called compiler directives.
- All such lines begin with the `#` character as the first non-white space character on the line and are of one of three types of directives:
 - Macro definitions
 - File inclusion
 - Conditional compilation

Compilation vs. Interpretation

- Macro- Syntax: `#define`
 - This macro defines constant value and can be any of the basic data types.
- Header file inclusion- Syntax: `#include <file_name>`
 - The source code of the file “file_name” is included in the main program at the specified place.
- Conditional compilation-
- Syntax: `#ifdef`, `#endif`, `#if`, `#else`, `#ifndef`
 - Set of commands are included or excluded in source program before compilation with respect to the condition.
- Other directives -Syntax: `#undef`
 - `#undef` is used to undefine a defined macro variable.

Compilation vs. Interpretation

//Demonstration of C preprocessor capabilities

```
//include directive, macros and
conditional compilation
#include<stdio.h>
#define x 10
//#define DEBUG 1
int a, b, c, d;
main()
{
    printf("Welcome to DPPL sessions\n");
    printf("Enter any two numbers \n");
    scanf("%d%d",&a, &b);
    c = a + b;
    c = c + x;
    printf("The sum of two numbers is %d \n ", c);

    #undef x
    #define x 20
    d = d + x;
    #ifdef DEBUG
        printf("debug:c = %d, d= %d\n", c, d);
    #endif
    printf("The sum is %d \n ", d);
}
```

Compilation vs. Interpretation

//Demonstration of C preprocessor capabilities

```
//include directive, macros and
conditional compilation
#include<stdio.h>
#define x 10
#define DEBUG 1
int a, b, c, d;
main()
{
    printf("Welcome to DPPL sessions\n");
    printf("Enter any two numbers \n");
    scanf("%d%d",&a, &b);
    c = a + b;
    c = c + x;
    printf("The sum of two numbers is %d \n ", c);

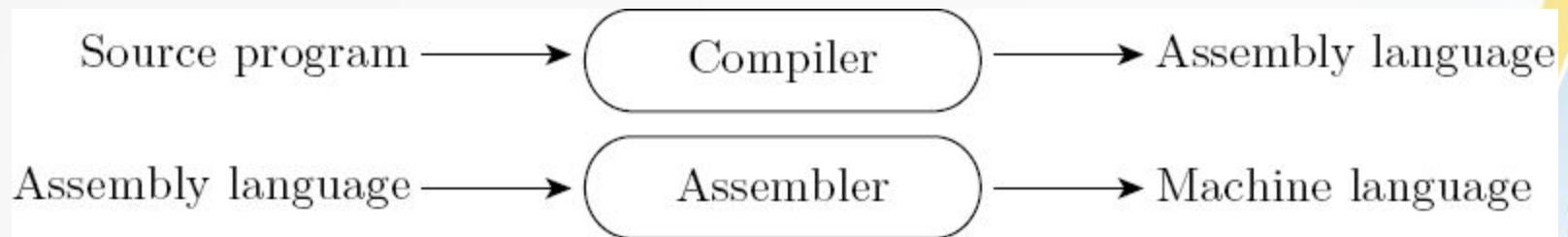
    #undef x
    #define x 20
    d = d + x;
    #ifdef DEBUG
        printf("debug:c = %d, d= %d\n", c, d);
    #endif
    printf("The sum is %d \n ", d);
}
```

Compilation vs. Interpretation

- Implementation strategies:

1. Post-compilation Assembly

- Facilitates debugging (assembly language easier for people)
- Isolates the compiler from changes in the format of machine language files (only assembler must be changed, is shared by many compilers)

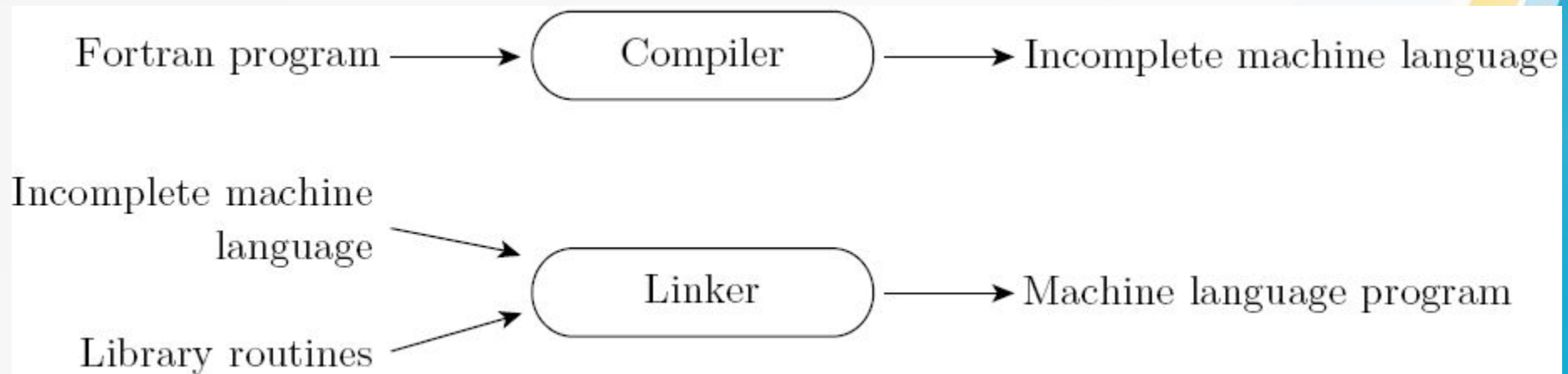


Compilation vs. Interpretation

- Implementation strategies:

- .Library of Routines and Linking

- Compiler uses a *linker* program to merge the appropriate *library* of subroutines (e.g., math functions such as sin, cos, log, etc.) into the final program:

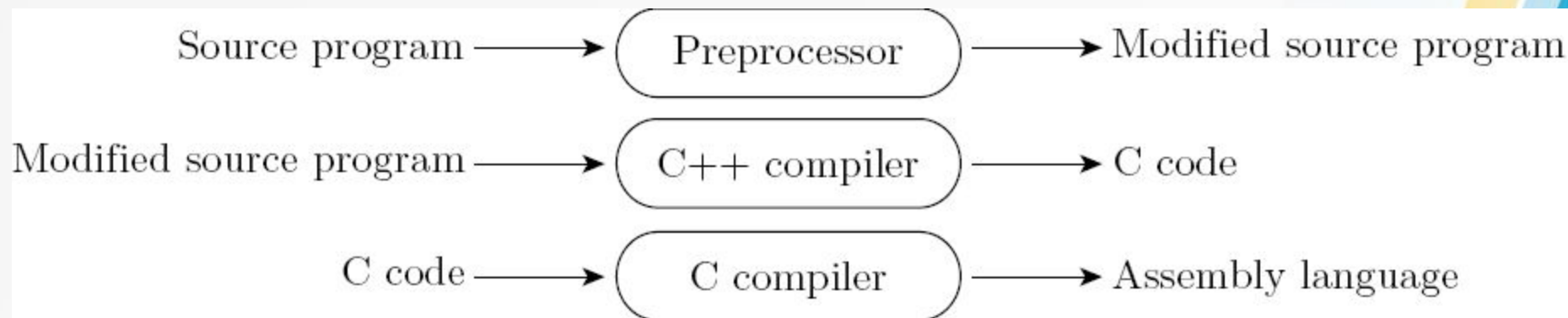


Compilation vs. Interpretation

- Implementation strategies:

- Source-to-Source Translation (C++)

- C++ implementations based on the early AT&T compiler generated an intermediate program in C, instead of an assembly language:



- Compilation entails semantic *understanding* of what is being processed; pre-processing does not.
 - A pre-processor will often let errors through.

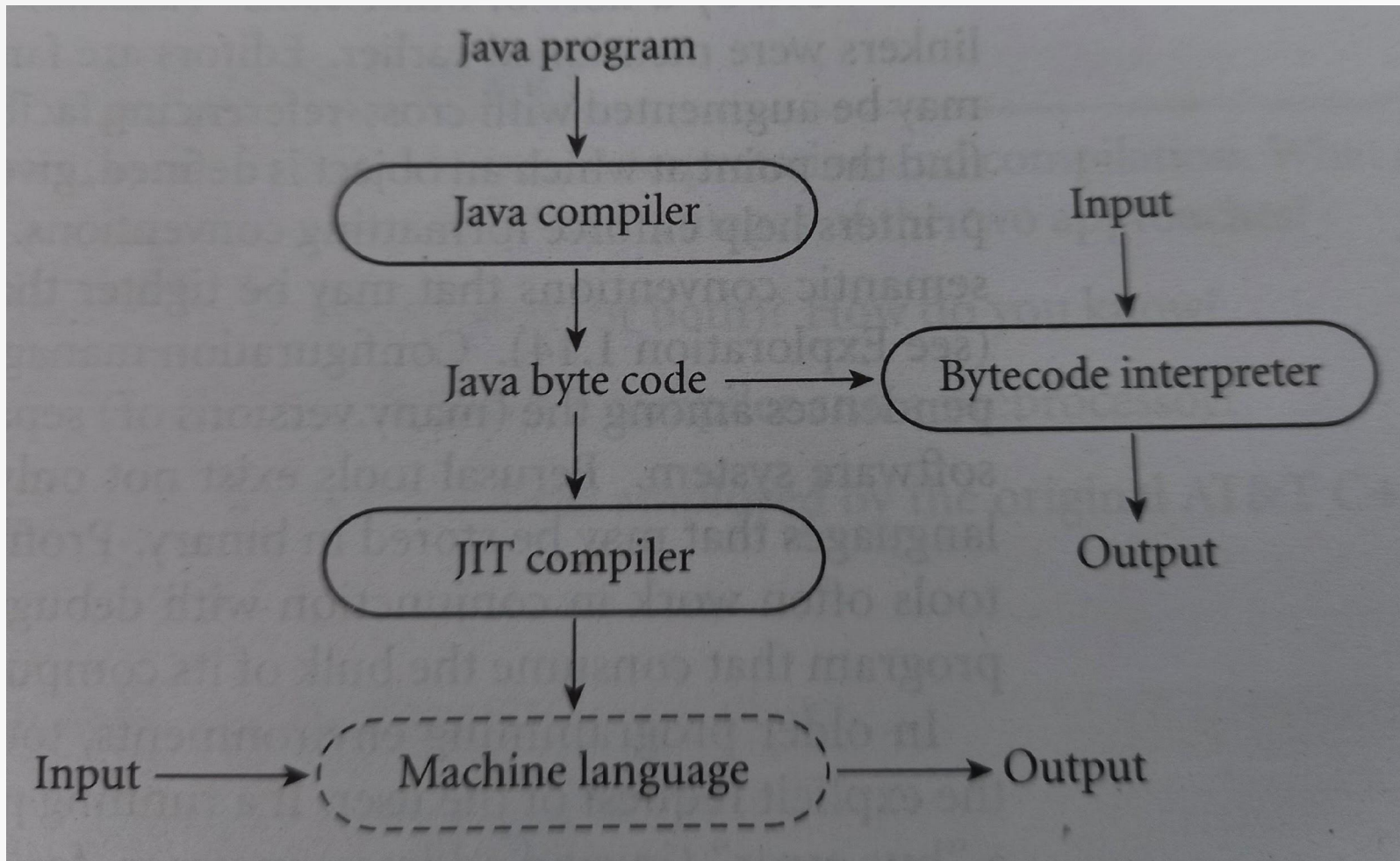
Compilation vs. Interpretation

- Implementation strategies:

- .-Dynamic and Just-in-Time Compilation

- In some cases a programming system may deliberately delay compilation until the last possible moment.
 - The Java language definition defines a machine-independent intermediate form known as *byte code*. Byte code is the standard format for distribution of Java programs.
 - The main C# compiler produces .NET Common Intermediate Language (CIL), which is then translated into machine code immediately prior to execution.

Just-in-Time Compilation Example



References

- Programming Language Pragmatics - By Michael Scott Elsevier -Third Edition